SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA0307	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	K.Kavyanjali	**Reg. No.:**	192324133
Section / Batch:	B.Tech-AI&DS	**Date:**	23/07/2026

Code of Conduct

I _am C. Nithisha(192424375) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K.Kavyanjali

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Annexure A – Application Based Questions

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.
Identify email addresses.
Extract mobile numbers.
Detect technical skills (Python, Java, SQL, Machine Learning, NLP).
Extract years of experience.
Generate a structured summary of the candidate's profile.
Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.
Perform prefix-based searches.
Perform suffix-based searches.
Search using partial keywords.
Perform case-insensitive searches.
Display all matching product names.
Generate a report showing the total number of matching products for each search.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.
Validate the institutional email address.
Validate the course code.
Validate the semester information.
Validate the student's mobile number.
Display appropriate validation messages for each field.
Generate a final registration status report indicating whether the registration is successful.

Q1: Regular Expressions – Resume Information Extraction

1. Understanding of Problem Context:

Many organizations receive a large number of resumes during recruitment. Reviewing every resume manually takes considerable time and may lead to mistakes. To simplify this process, an automated system can be developed using Python Regular Expressions (Regex). The system extracts important candidate details such as name, email address, contact number, technical skills, and work experience. It then checks whether the applicant satisfies the company's eligibility conditions and prepares a summary for HR.

2.Application of Concepts:

Concept Used

- Python Programming
- Regular Expressions (Regex)
- Text Processing
- Pattern Matching
- Information Extraction

Regex Patterns Used

Field	Regex Pattern
Name	Name:\s*(.*)
Email	[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}
Mobile Number	[6-9]\d{9}
Skills	Python|Java|SQL|Machine Learning|NLP
Experience	(\d+)\s+years

Python Functions Used

- re.search()
- re.findall()
- group()
- print()

These concepts are applied to identify and extract candidate information from the resume text efficiently.

3. Problem-Solving Approach:

Algorithm

Step 1: Import the re module.

Step 2: Store the resume information in a string.

Step 3: Search for the candidate's name.

Step 4: Extract email address.

Step 5: Find the mobile number.

Step 6: Identify technical skills.

Step 7: Extract years of experience.

Step 8: Display all extracted information..

Step 9: Check whether:

Verify whether the candidate has Python skill and at least two years of experience.

Step 10: If both conditions are satisfied, display **"Eligible "**; otherwise display **"Not Eligible"**.

Step 11: End the program.

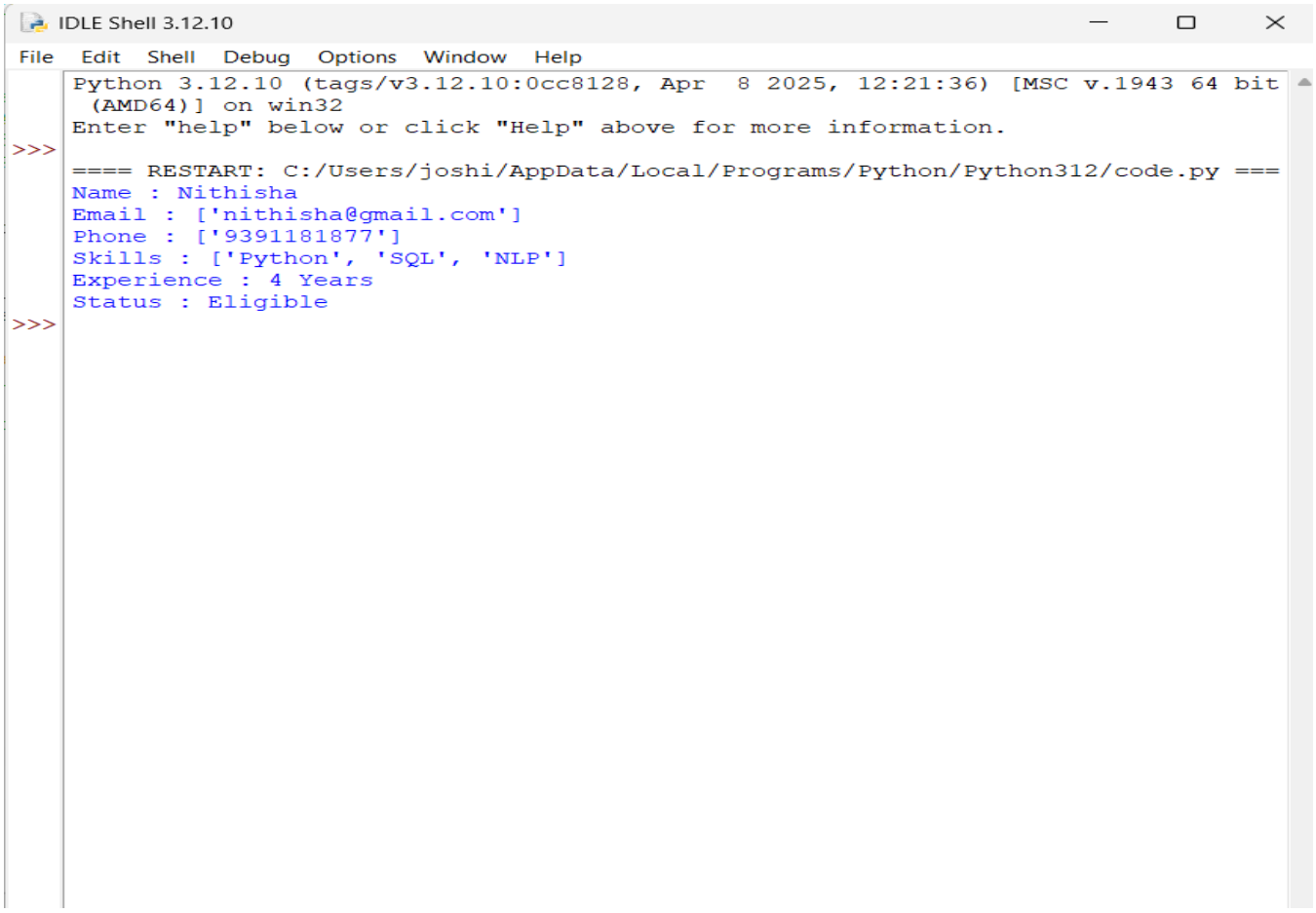
4.Accuracy of Solution:

Code:

```
import re
resume = """
Name: Nithisha
Email: nithisha@gmail.com
Phone: 9123456789
Skills: Python, SQL, NLP
Experience: 4 Years
"""
name = re.search(r"Name:\s*(.+)", resume)
email = re.findall(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}", resume)
phone = re.findall(r"[6-9]\d{9}", resume)
skills = re.findall(r"Python|Java|SQL|Machine Learning|NLP", resume)
exp = re.search(r"(\d+)\s+Years?", resume)
print("Name :", name.group(1))
print("Email :", email)
print("Phone :", phone)
print("Skills :", skills)
print("Experience :", exp.group(1), "Years")
if "Python" in skills and int(exp.group(1)) >= 2:
```

```
    print("Status : Eligible")
else:
    print("Status : Not Eligible")
```

Output:



Test Cases

Experience	Python Skill	Result
4 Years	Yes	Eligible
1 Year	Yes	Not Eligible
6 Years	No	Not Eligible

5. Clarity:

Output Explanation

The program scans the resume using Regular Expressions and extracts important information including candidate name, email, mobile number, technical skills, and work experience. It then verifies whether the candidate has Python knowledge and at least two years of work experience. If both conditions are satisfied, the candidate is considered eligible for recruitment.

Conclusion

The Resume Information Extraction System automates candidate screening using Python Regular Expressions. It improves efficiency, reduces manual effort, and helps recruiters quickly identify suitable candidates based on predefined eligibility criteria.

Q2: Regular Expressions – Product Search System

1. Understanding of Problem Context:

Online shopping websites contain thousands of products, making it difficult for customers to locate items quickly. A product search system using Python Regular Expressions can improve search efficiency by allowing users to search using exact words, prefixes, suffixes, partial words, and case-insensitive matching. The system displays all matching products and provides a summary report of the search results.

2. Application of Concepts:

Concepts Used

- Python Programming
- Regular Expressions (Regex)
- Pattern Matching
- String Searching
- Case-Insensitive Search

Regex Patterns Used

Search Type	Regex Pattern
Exact Search	^keyword$
Prefix Search	^keyword
Suffix Search	keyword$
Partial Search	keyword
Case-Insensitive Search	re.IGNORECASE

Python Functions Used

- re.search()
- re.match()
- re.IGNORECASE
- len()

These concepts are used to perform flexible product searches and count the matching products.

2. Problem-Solving Approach:

Algorithm

Step 1: Import the re module.

Step 2: Create a list of product names.

Step 3: Read the search keyword from the user.

Step 4: Perform an exact search using Regex.

Step 5: Perform a prefix-based search.

Step 6: Perform a suffix-based search.

Step 7: Perform a partial keyword search.

Step 8: Perform a case-insensitive search.

Step 9: Display all matching products.

Step 10: Count and display the total number of matching products for each search type.

Step 11: End the program.

4.Accuracy of Solution:

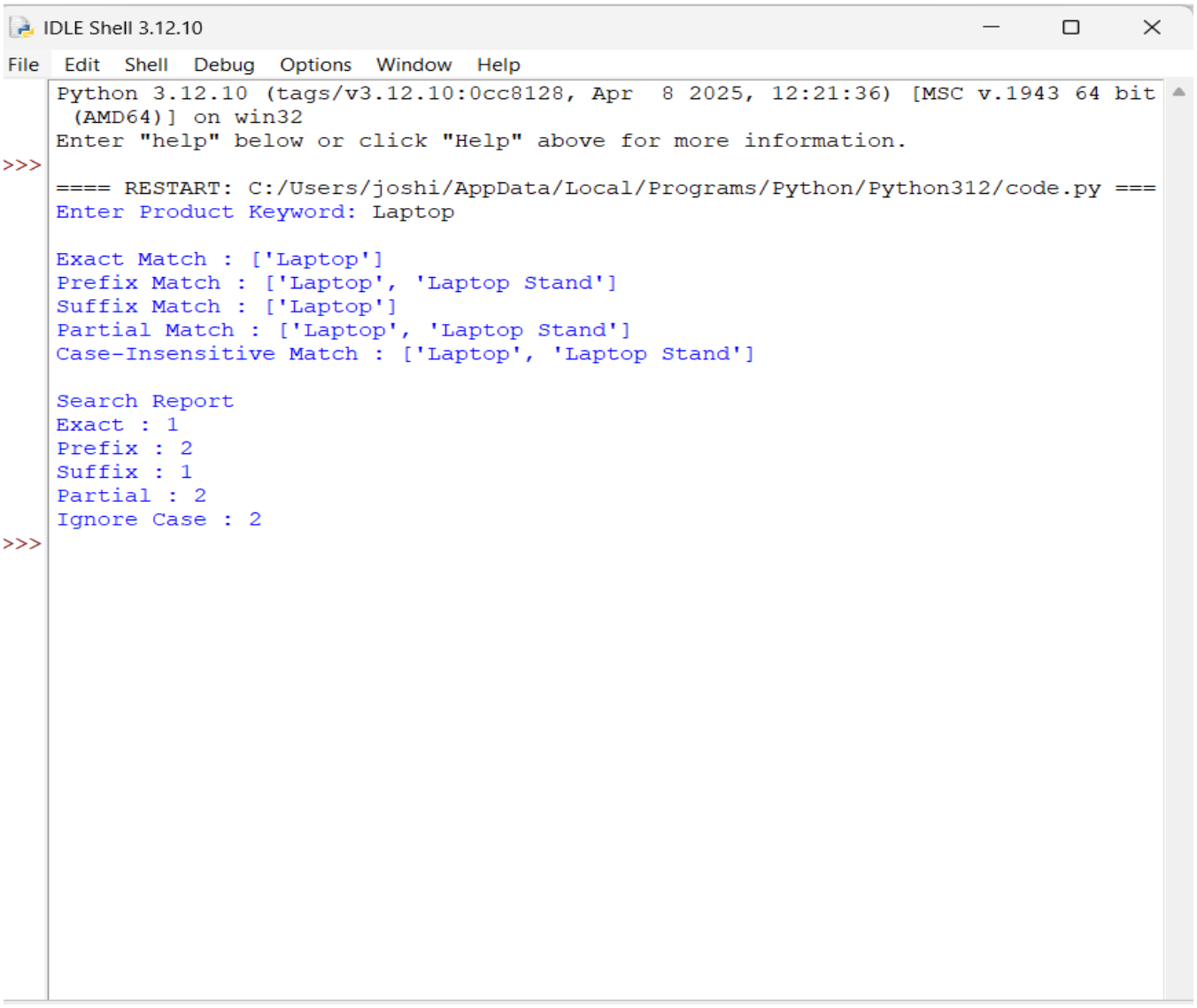
Code:

```
import re
products = [
    "Laptop",
    "Laptop Stand",
    "Wireless Mouse",
    "Gaming Mouse",
    "Keyboard",
    "USB Keyboard",
    "Smartphone",
    "Smart Watch",
    "Bluetooth Earbuds",
    "Speaker"
]
keyword = input("Enter Product Keyword: ")

exact = [p for p in products if re.fullmatch(keyword, p)]
prefix = [p for p in products if re.match(keyword, p)]
suffix = [p for p in products if re.search(keyword + r"$", p)]
partial = [p for p in products if re.search(keyword, p)]
ignore_case = [p for p in products if re.search(keyword, p, re.IGNORECASE)]
print("\nExact Match :", exact)
```

```
print("Prefix Match :", prefix)
print("Suffix Match :", suffix)
print("Partial Match :", partial)
print("Case-Insensitive Match :", ignore_case)
print("\nSearch Report")
print("Exact :", len(exact))
print("Prefix :", len(prefix))
print("Suffix :", len(suffix))
print("Partial :", len(partial))
print("Ignore Case :", len(ignore_case))
```

Output:



5. Clarity:

Output Explanation

The program stores a list of product names and accepts a search keyword from the user. Using Regular Expressions, it performs five different types of searches:

- **Exact Search:** Finds products whose name exactly matches the keyword.
- **Prefix Search:** Finds products that start with the keyword.
- **Suffix Search:** Finds products that end with the keyword.
- **Partial Search:** Finds products containing the keyword anywhere in the name.
- **Case-Insensitive Search:** Finds products regardless of uppercase or lowercase letters.

Finally, the program displays all matching product names and generates a report showing the total number of matches for each search type.

Conclusion

The Product Search System successfully demonstrates the use of Python Regular Expressions for flexible keyword searching. It improves product discovery by supporting multiple search methods and provides an accurate report of matching products. This approach enhances customer experience and makes product searching more efficient.

Q3: Regular Expressions – University Registration System

1. Understanding of Problem Context:

Educational institutions require students to enter correct registration details before course enrollment. Incorrect information may lead to registration failures and data inconsistencies. A validation system using Python Regular Expressions helps verify student details such as register number, institutional email, course code, semester, and mobile number before accepting the registration.

2. Application of Concepts:

Concepts Used

- Python Programming
- Regular Expressions
- Input Validation
- Pattern Matching
- Conditional Statements

Regex Patterns Used

Field	Regex Pattern	Example
Register Number	^\d{12}$	312223104001
Institutional Email	^[A-Za-z0-9._%+-]+@saveetha.com$	student@saveetha.com
Course Code	^[A-Z]{3}\d{2}$	DSA03
Semester	^[1-8]$	5
Mobile Number	^[6-9]\d{9}	9391181877

Python Functions Used

- re.match()
- input()
- if-else
- print()

These concepts are used to validate each input field according to the university's registration rules.

3. Problem-Solving Approach:

Algorithm

Step 1: Import the re module.

Step 2: Read the student's register number.

Step 3: Read the institutional email address.

Step 4: Read the course code.

Step 5: Read the semester.

Step 6: Read the mobile number.

Step 7: Validate each field using the appropriate Regular Expression.

Step 8: Display "Valid" or "Invalid" for every field.

Step 9: If all fields are valid, display **"Registration Successful"**.

Step 10: Otherwise, display **"Registration Failed"**.

Step 11: End the program.

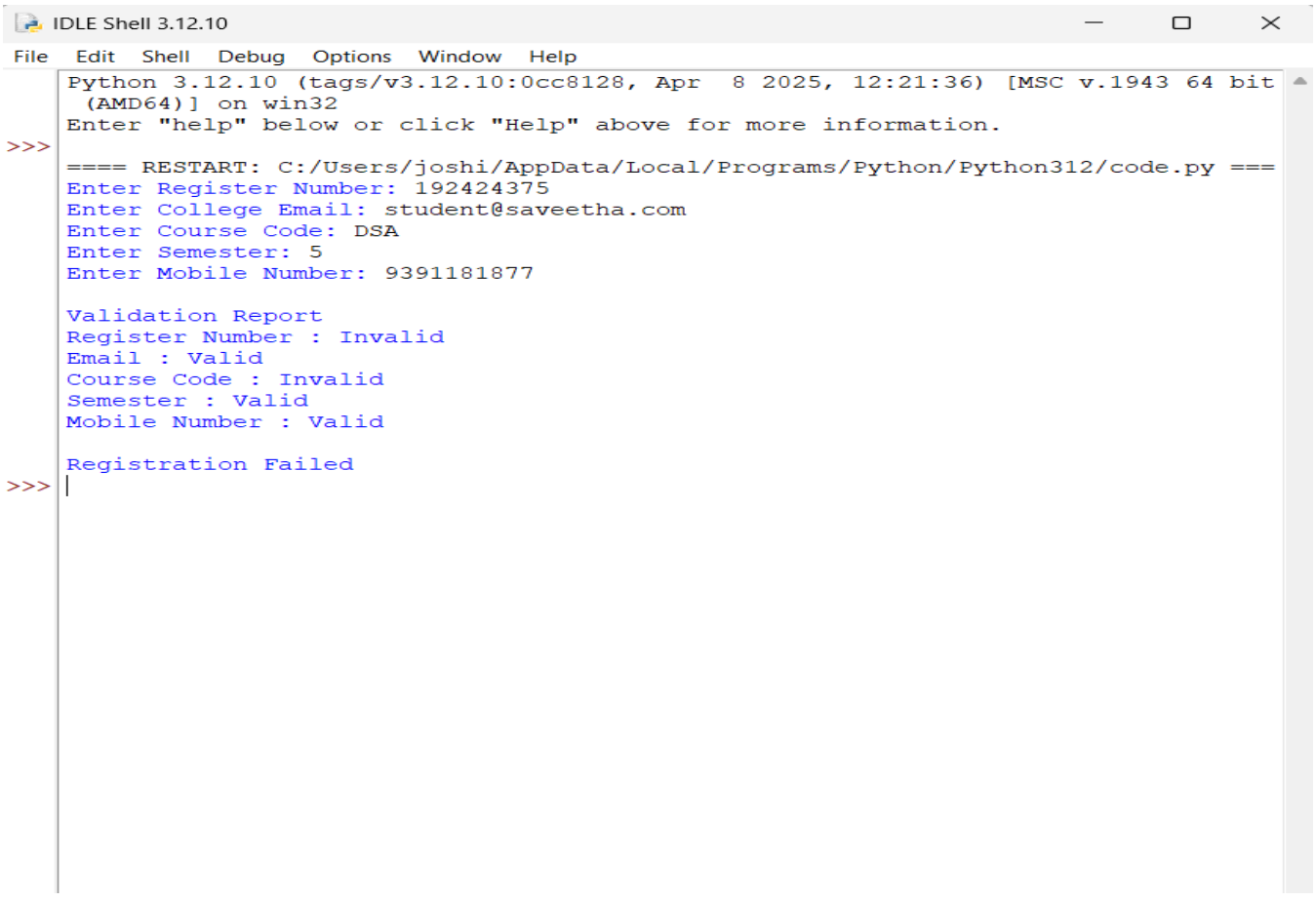
4. Accuracy of Solution:

Code:

```
import re
reg = input("Enter Register Number: ")
email = input("Enter College Email: ")
course = input("Enter Course Code: ")
semester = input("Enter Semester: ")
mobile = input("Enter Mobile Number: ")
```

```
valid_reg = bool(re.match(r"^\d{12}$", reg))
valid_email = bool(re.match(r"^[A-Za-z0-9._%+-]+@saveetha\.com$", email))
valid_course = bool(re.match(r"^[A-Z]{3}\d{2}$", course))
valid_sem = bool(re.match(r"^[1-8]$", semester))
valid_mobile = bool(re.match(r"^[6-9]\d{9}$", mobile))
print("\nValidation Report")
print("Register Number :", "Valid" if valid_reg else "Invalid")
print("Email :", "Valid" if valid_email else "Invalid")
print("Course Code :", "Valid" if valid_course else "Invalid")
print("Semester :", "Valid" if valid_sem else "Invalid")
print("Mobile Number :", "Valid" if valid_mobile else "Invalid")
if all([valid_reg, valid_email, valid_course, valid_sem, valid_mobile]):
    print("\nRegistration Successful")
else:
    print("\nRegistration Failed")
```

Output:



5. Clarity:

Output Explanation

The program validates every piece of information entered by the student using Regular Expressions. Each input is checked against the required format, and the validation result is displayed immediately. If all fields satisfy the validation rules, the registration process is completed successfully; otherwise, the registration is rejected.

Conclusion

The University Registration Validation System effectively uses Python Regular Expressions to verify student information before enrollment. It minimizes data entry errors, improves the reliability of the registration process, and ensures that only valid information is stored in the system.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____